Operating Systems

Complete Study Notes

Topics Covered:

1. Introduction to Operating Systems
2. Process Management and CPU Scheduling
3. Threads and Concurrency
4. Memory Management and Virtual Memory
5. File Systems and Deadlock

1. Introduction to Operating Systems

An Operating System (OS) is system software that acts as an intermediary between computer hardware and application programs. It manages hardware resources like the CPU, memory, storage, and input/output devices, providing a stable and consistent environment for programs to run.

Key goals of an OS include: resource allocation and management, hardware abstraction, user interface provision, and system security enforcement.

Types of Operating Systems: Batch OS processes jobs in batches without user interaction. Time-Sharing OS allows multiple users to use a system simultaneously. Real-Time OS guarantees responses within strict time constraints (used in embedded systems). Distributed OS manages a group of networked computers as a single system.

2. Process Management and CPU Scheduling

A process is defined as a program in execution. It is the basic unit of work in a modern operating system. Each process has its own address space, code, data, and stack.

Process States: A process moves through the following states during its lifetime: New (being created), Ready (waiting for CPU), Running (instructions executing), Waiting/Blocked (waiting for I/O), and Terminated (execution finished).

The Process Control Block (PCB) stores all information about a process: process ID, process state, program counter, CPU registers, memory management information, and I/O status.

CPU Scheduling Algorithms:
First-Come First-Served (FCFS) is a non-preemptive algorithm that serves processes in the order they arrive. It is simple but suffers from the Convoy Effect.
Shortest Job First (SJF) selects the process with the smallest burst time, achieving minimum average waiting time but requiring advance knowledge of burst times.
Round Robin (RR) is a preemptive algorithm that assigns a fixed time quantum to each process cyclically, making it ideal for time-sharing systems.
Priority Scheduling assigns CPU to the highest-priority process, but can cause starvation of low-priority processes, which is resolved using Aging.

3. Threads and Concurrency

A thread is defined as the smallest unit of execution within a process, often called a lightweight process. Multiple threads within the same process share code, data sections, and OS resources, but each maintains its own program counter, registers, and stack.

Benefits of Multithreading: Responsiveness (a process can continue even if part of it is blocked), Resource Sharing (threads share memory and resources of the parent process), Economy (creating threads is cheaper than creating processes), and Scalability (threads can run in parallel on multi-core CPUs).

Types of Threads: User-Level Threads are managed entirely by a user-space thread library without OS knowledge. Kernel-Level Threads are managed directly by the OS, allowing true parallel execution on multi-core systems.

Concurrency Problems: Race Condition occurs when multiple threads access shared data and the result depends on execution order. Critical Section is a segment of code that must not be executed by more than one thread at a time. Mutex (Mutual Exclusion) is a locking mechanism that ensures only one thread enters the critical section. Semaphore is a synchronization tool that uses a counter to control access to shared resources. Deadlock is a situation where a set of processes are each waiting for a resource held by another, causing a permanent block.

4. Memory Management and Virtual Memory

Memory Management is the functionality of an OS responsible for managing the computer's primary memory (RAM). The OS tracks which parts of memory are in use and which are free, allocates memory to processes when needed, and reclaims it when a process finishes.

Paging is a memory management scheme that eliminates the need for contiguous physical memory allocation. Physical memory is divided into fixed-size blocks called Frames, and logical memory is divided into blocks of the same size called Pages. A Page Table maps logical page numbers to physical frame numbers.

Segmentation divides memory into variable-sized segments corresponding to logical units like functions, arrays, or stacks. Each segment has a base address and a limit.

Virtual Memory is a technique that allows the execution of processes that may not be completely in memory. It extends the apparent size of RAM by using secondary storage (disk) as an extension of main memory. Demand Paging is the most common virtual memory implementation: pages are loaded into memory only when they are needed, reducing memory usage. A Page Fault occurs when a required page is not in memory, triggering the OS to load it from disk. Thrashing refers to excessive page swapping, where the OS spends more time swapping pages than executing processes.

5. File Systems and Deadlock

A File System is a method and data structure used by the OS to organize, store, retrieve, and manage files on a storage device. Common file systems include FAT32, NTFS (Windows), ext4 (Linux), and APFS (macOS).

File operations supported by an OS: Create, Read, Write, Append, Seek, Delete, and Truncate. File attributes include name, type, location, size, protection permissions, and timestamps.

Directory Structures: Single-Level Directory has all files in one directory. Two-Level Directory has a separate directory for each user. Tree-Structured Directory allows a hierarchical directory structure. Acyclic-Graph Directory allows sharing of subdirectories and files.

Deadlock is a situation where a set of processes are blocked because each process is holding a resource and waiting to acquire a resource held by another process in the set. The four necessary conditions (Coffman Conditions) for deadlock are:
1. Mutual Exclusion: At least one resource must be non-shareable.
2. Hold and Wait: A process holds a resource while waiting for more.
3. No Preemption: Resources cannot be forcibly taken from a process.
4. Circular Wait: A closed circular chain of processes each waiting for a resource held by the next.

Deadlock Handling Strategies: Deadlock Prevention eliminates one of the four conditions. Deadlock Avoidance uses algorithms like Banker's Algorithm to ensure the system never enters an unsafe state. Deadlock Detection allows the system to enter a deadlocked state and then recover. Deadlock Ignorance (Ostrich Algorithm) ignores the problem ? used when deadlocks are rare and costly to handle.